On Engineering AI Systems
that Endure the Bitter Lesson

Omar Khattab

You engineer AI *software*.

Every week, a **new LLM** comes out.

With new guidance on prompting quirks!

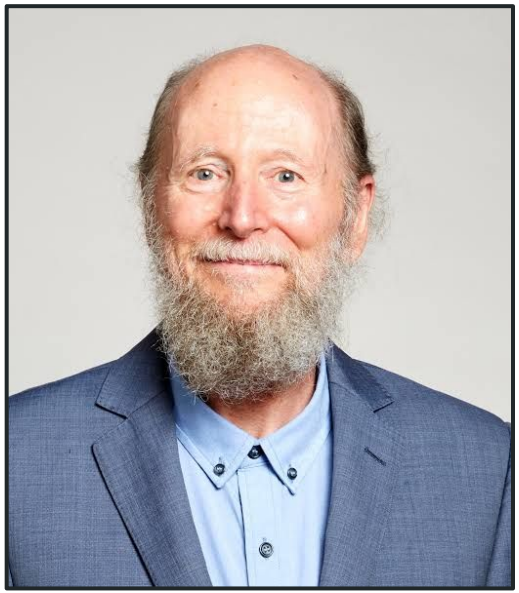
Every *day*, a **new learning algorithm** or **inference strategy** promises to finally unlock great reliability.

Do you have to scramble every week forever?
Or, uh..., will you even *get* to scramble for long?

The Bitter Lesson

Rich Sutton

March 13, 2019



Seventy years of AI teach us that:

When we leverage domain knowledge, we build *complicated methods* that don't scale.

But *general methods that scale*, i.e. search and learning, work best!

(*Search* here is what we'd call inference-time scaling.)

Disclaimer: I don't speak for Sutton. He's probably never heard of me and he probably doesn't think of the bitter lesson the way you do—e.g., Sutton thinks standard SGD and DNNs are essentially too simple for the kind of intelligence he's interested in!

But if leveraging domain knowledge is that bad, what exactly is *AI "engineering"* supposed to be all about?

Answer: Sutton is talking about *maximizing intelligence*, or the ability to quickly figure things out in a new environment.

But... We don't program *software* because we lack AGI. We have unreliable general intelligence everywhere!

We program software because we want **reliable systems**. Reliable systems *subtract* agency in the right places—for **robust control**.

Takeaway 1: Scaling “search” and “learning” works best for AI. In building AI *systems*, you have to carefully engineer ***what to search for*** and ***what are you learning for***.

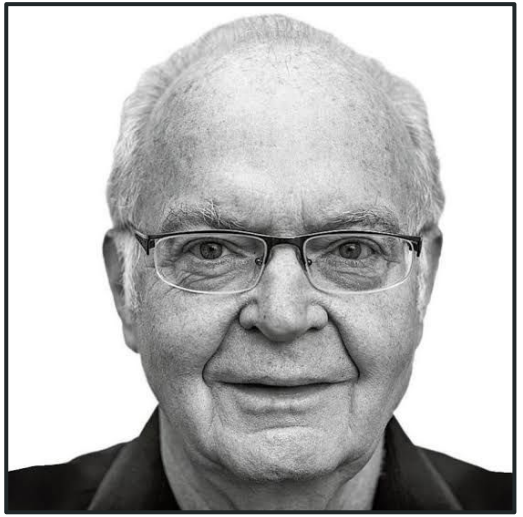
Think of it this way: If I say “packing more transistors is what works best for computer chips”, it says nothing about software. You still need to engineer your actual *software*!

Hmm… “complicated methods get in the way of scaling”

Structured Programming with go to Statements

DONALD E. KNUTH

Stanford University, Stanford, California 94305



“Premature optimization is the root of all evil.”

This is the bitter lesson for *software*, including **AI Software**.

```
    x2 = number * 0.5F;
    i  = 0x5f3759df - ( i >> 1 );                    // what the fuck?
    y  = * ( float * ) &i;
    y  = y * ( threehalfs - ( x2 * y * y ) );   // 1st iteration
//  y  = y * ( threehalfs - ( x2 * y * y ) );   // 2nd iteration,
```

Not the most future-proof program possible!

Premature optimization is the [square?] root of all evil.
But… what counts as *premature*?

No strategy will “just work” forever, but I wanted to reflect on the approach I took in designing ColBERT (‘19) and DSPy (‘22).

Both have maintained their fundamental design since BERT and text-davinci-002 and up to o4-mini, and are somehow larger now than ever. **What gives?! How did they manage to escape?**

Hypothesis: Premature optimization happens *iff* you hand-engineer stuff at a lower *level of abstraction than necessary*!

If you *want* a SQRT, say "I want a SQRT". Don't start doing crazy bit manipulation that appease your specific machine—unless you exhausted the systematic options!

But wait a second: *do you want* a SQRT? Or is there an even higher goal that's getting shadowed here?

Tighter coupling than necessary is known to be bad for software. This needs to become a basic principle in Applied ML (and in prompt engineering) too.



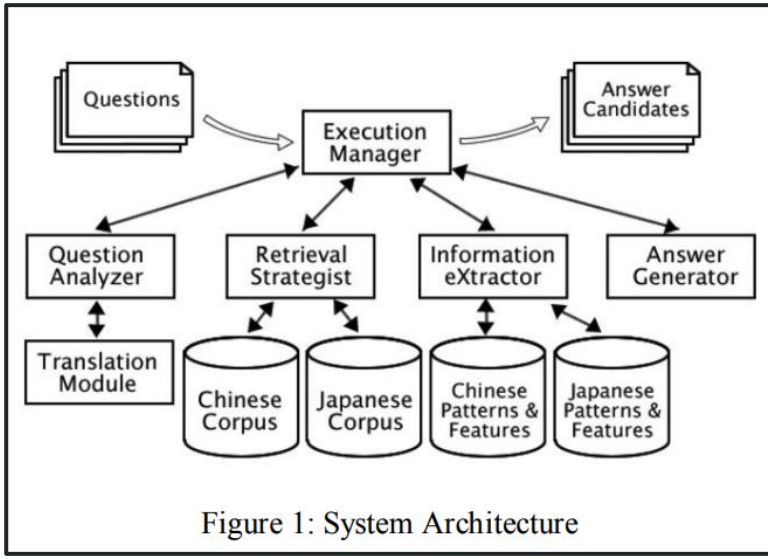


Figure 1: System Architecture

Modular Approach to Error Analysis and Evaluation for Multilingual Question Answering

2006 paper!

Hideki Shima, Mengqiu Wang, Frank Lin, Teruko Mitamura

That 2006 diagram is NOT *the* perfect architecture by any stretch.

But if you *could* swap each module's implementation out for the modern alternative, you'd probably get a solid piece of AI software…

The problem is that you typically can't. Applied ML usually isn't expressed at the right level of abstraction.

Similarly, a prompt is a horrible abstraction for *programming*.

A *stringly*-typed canvas that entangles *what* your **task definition** is with half-baked, overfitted decisions for:

1. how you want to **hill-climb the wording**, structure, examples, or environment-dependent nuances for *one* specific LLM!
2. how the **inference-time strategy** influences your instructions AND examples (e.g., adjusting from CoT reasoning to a tool-use Agent)
3. how you decide to **format your input values** (and in ways that mix instructions with data, reducing the scope of possible optimizations!)
4. how you specify the **format for parsing outputs** and how you actually handle the parsing/retries.

You want to write a human-readable Spec but you're hard-coding:

1. **"Here is the list of tools. DO NOT IGNORE THE…."**
2. **"Generate <think> in XML format."**
3. **"But <answer> with JSON structure."**
4. **"You are Prof. Einstein, a wise expert in the field of…."**
5. **"I will tip you $1000."**

Trusty old **Separation of Concerns...**

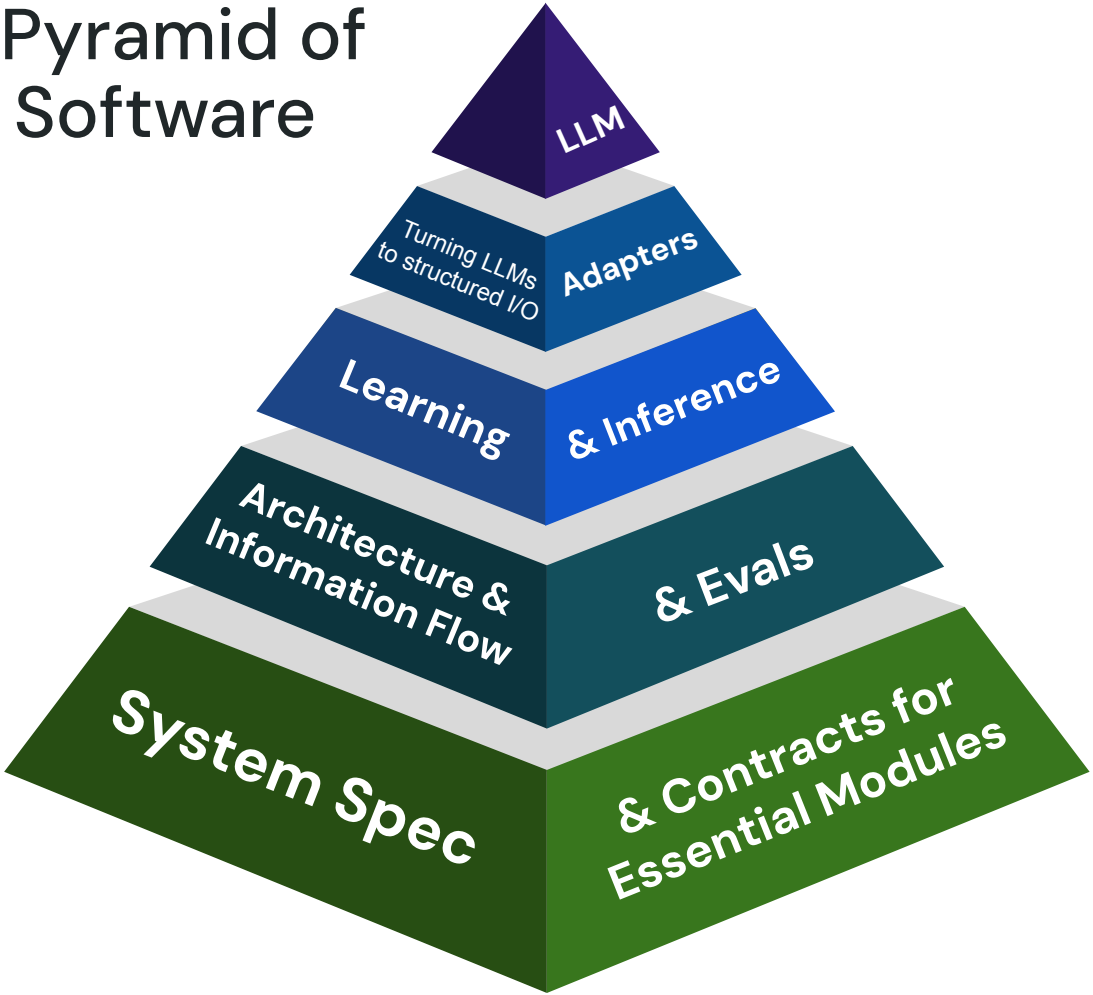
- Your job is to invest in your *actual* spec. Some things are best expressed as:
  - **Natural language definitions**, like guidelines or I/O *schemas*!
  - **Evals**, like the distribution of data inputs you get.
  - **Code**, like the tools, essential control flow, and function composition.

- You have to do express those in a way that is *decoupled* from:
  - new **models**
  - new **inference strategies** (Sutton's "search")
  - new downstream **learning algorithms**: RL methods, prompt optimizers

Takeaway 2:

- Invest in defining things *specific* to your AI systems.
- Invest in decoupling your system from the lower-level, swappable pieces, because they'll expire faster than ever!

The Pyramid of LLM Software



Only incidental to your AI system

Engineering by hand is wasted! **Try many, interchangeably!**

Each choice will expire fast!

Fundamental and specific to your AI system

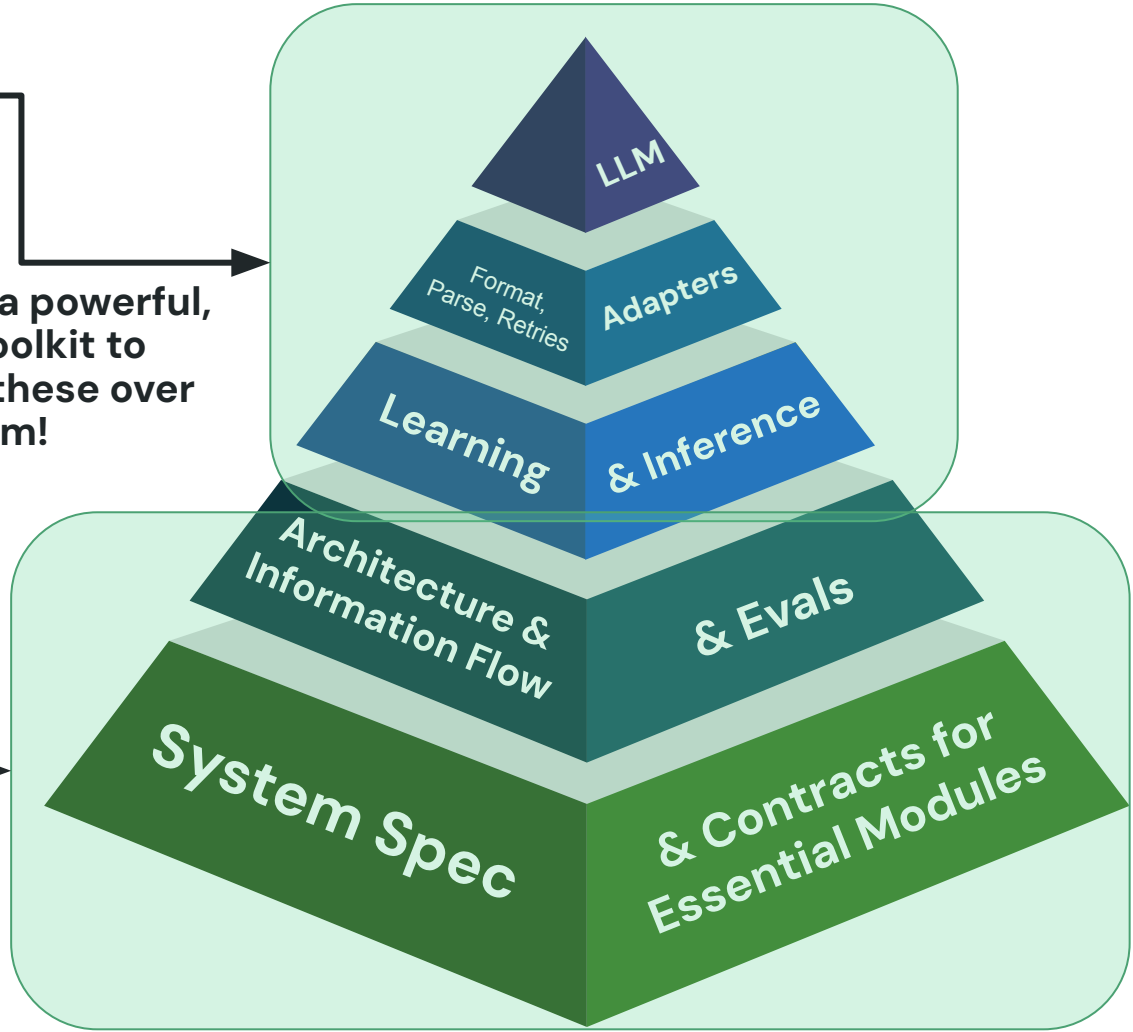
Engineering by hand pays off! **Invest!**

DSPy

Declarative
Self-improving
Python

Gives you a powerful, evolving toolkit to compose these over your system!

Gives you an embedded language (in Python) to express this.



The one concept that makes DSPy work: declarative **Signatures** as a first-class construct.

```python
class ExtractInfo(dspy.Signature):
    """Extract structured information from a document."""  # Task definition.

    # Input structure
    frontpage: dspy.Image = dspy.InputField()
    text: str = dspy.InputField()
    metadata: dict[str, Any] = dspy.InputField(desc="variable metadata about the text")

    # Output structure
    title: str = dspy.OutputField(desc="a fitting title of your creation for the text")
```

Notice that this says nothing about:

1. **Inference:** CoT? Agent w/ tools? Iterative Reflection? Recursive Processing?
2. **Aligning the LLM** via instructions, examples, or weights w/ your downstream objective.
3. LM-specific formatting of inputs and outputs. (No, JSON is not always right!)
4. Bad API design from LLM Providers, e.g. special treatment of Tools, JSON, Images, …

Summary. We can't predict the future, but **the *least* you can do is to avoid hand-engineering at TODAY's wrong level of abstraction**.

Among your safest bets are that Foundation Models won't any time soon:

- **Read the spec off of your mind.**
- **Magically collect all the structure & tools specific to your application!**

Hence: Invest in (1) Signatures, (2) *Essential* Control Flow & Tools, (3) Evals.

And ride the wave of swappable: (1) Models, (2) Modules, (3) Optimizers.

“But, Omar, we all now know that Abstractions are bad. I got burned from X or Y last year, or the year before.”

Poor abstractions *are* bad. People without the requisite conceptual model won’t build good ones. **But computer science *is* nothing but problem decomposition and abstraction.**

For example, a **Signature** is a timeless object: it’s a human-readable, type-structured contract of a function.

Takeaway 3: The community needs to think about fundamental abstractions, from the bottom up.

- The underlying algorithms/models will evolve and change, but a fundamental abstraction can outlive many methods.
- Start at a very high level: Only go down to a lower level of abstraction if the higher levels have actively failed you!

1. Scaling “search” and “learning” is what works for AI. But for AI *systems*, you have to carefully engineer *what* to search for and learn towards.

2. Invest in defining things *specific* to your AI systems. Invest in decoupling your system from the lower-level, swappable pieces. They’ll expire fast! (There’s one framework for this: dspy.ai --- three years into development.)

3. The community needs to think about fundamental programming models of *what* needs to be expressed. (MCP might be one such step, for tools!) And to consolidate efforts to make their implementations work as reliably and universally as traditional languages, runtimes, and compilers.

Without these, we’re not serious about AI *engineering* as a field.